

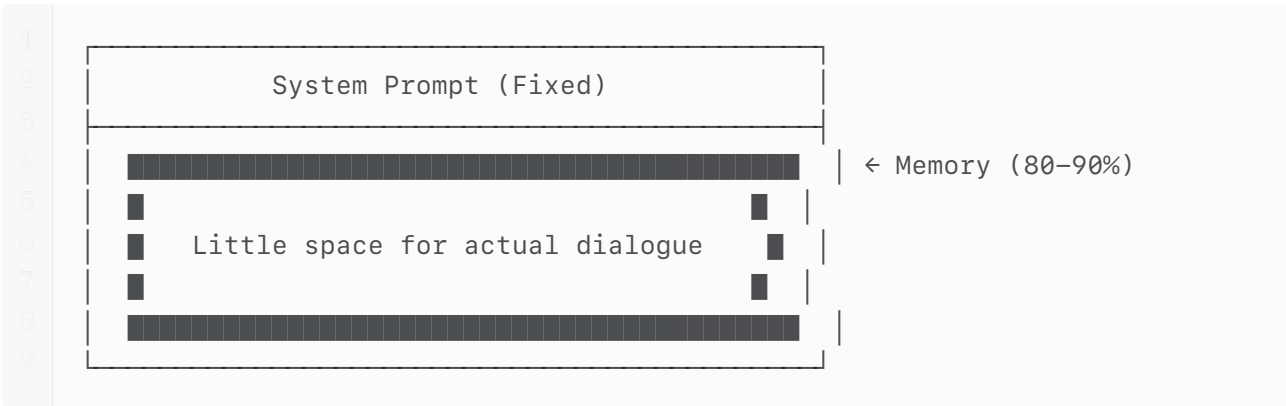
Advanced Memory Index Pattern for Hermes Agent

A two-layer memory optimization strategy that reduces token consumption by 30-50% while preserving all operational rules.

Why This Pattern Exists

The Memory Bottleneck in Long-Running Agents

Hermes Agent injects both **Memory** and **User Profile** into the System Prompt at every conversation turn. As configurations, preferences, API credentials, and workflow rules accumulate, these sections can consume **80-90%** of the available context window.



According to [Hermes Agent's official documentation](#), context files like `AGENTS.md` are automatically read every session. However, when operational rules exceed the context window, the agent faces a critical trade-off: **remember everything and lose reasoning capacity, or forget rules and lose capabilities.**

The Core Paradox

AI needs to know rules exist, but doesn't need to see full rule text every turn.

Consider a typical external tool configuration:

```
1  # This 200-character block is injected EVERY turn, even when no
2  translation is needed
3  - Tool: External Translation CLI
4    Path: /opt/tools/translator/
5    Command: translate "text" --to en
6    API Key: sk-xxxxxxxxxxxxxxxxxxxxxx
7    App ID: app-xxxxxxxxxxxx
8    Usage: translate "Hello world" → "你好世界"
9    Notes: Supports 200+ languages, auto-detect source language
```

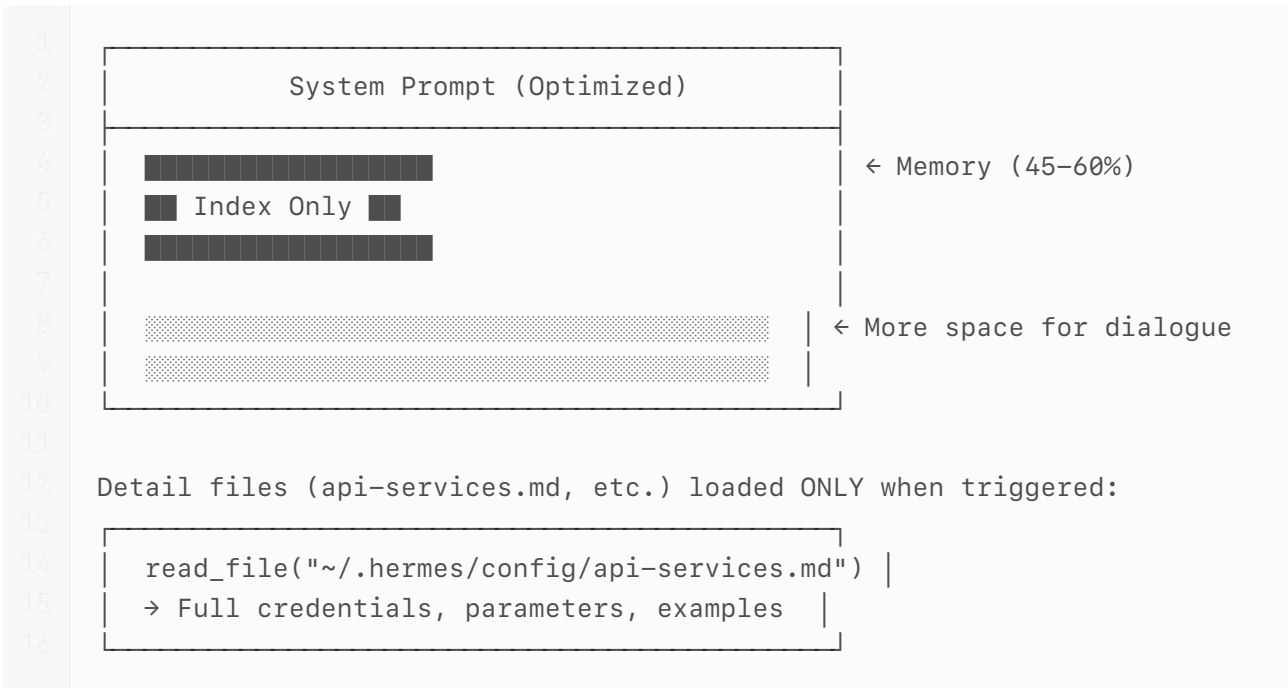
Reality: Translation is requested in <5% of conversations, but the full config is loaded 100% of the time.

Solution Overview

The Two-Layer Architecture

Split each rule into two layers:

Layer	Location	Content	Injection Frequency
Index	Memory	Trigger condition + tool reference	Every turn
Detail	External file	Full credentials, parameters, examples	On-demand only



Core Principle

1	Trigger Condition + Tool Index	→ Stay in Memory (injected every turn, one line is enough)
2	Full Details	→ Store in File (loaded on-demand)

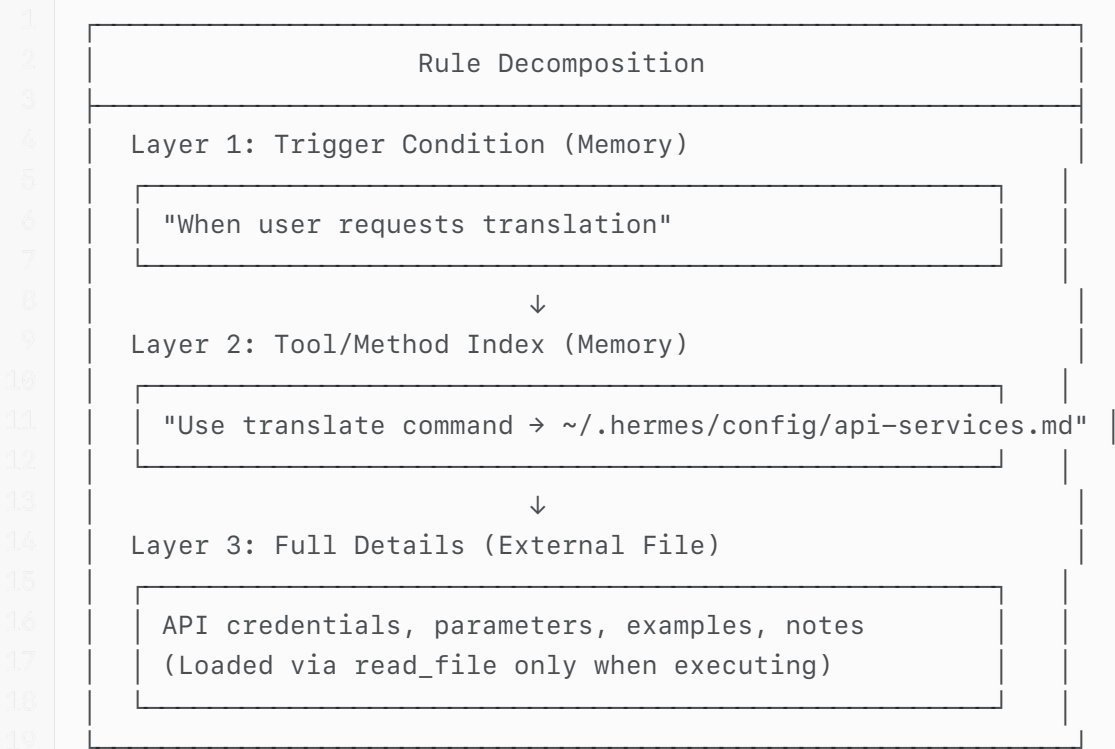
Example transformation:

```
1 # Before (200 chars, injected every turn)
2 - External Translation CLI at /opt/tools/translator/
3   API Key: sk-xxxxxxxxxxxxxxxxxxxx, App ID: app-xxxxxxxxxxxx
4   Command: translate "text" --to en, supports 200+ languages
5
6 # After (65 chars, injected every turn)
7 + When translation needed → use translate command [→ ~/.hermes/config/
  api-services.md]
```

Technical Architecture

Three-Layer Rule Structure

Each operational rule consists of three layers:



Index Format Specification

Single Rule Format

```
1 <Trigger Condition> → use <Tool/Command> [→ <Detail File Path>]
```

Examples:

```
1 Translation needed → use translate command [→ ~/.hermes/config/api-
2 services.md]
3 Image with text received → use ocr command [→ ~/.hermes/config/api-
4 services.md]
5 Weather query → use weather command [→ ~/.hermes/config/api-services.md]
6 Text-to-speech needed → analyze emotion first, then use tts [→
7 ~/.hermes/config/workflow-preferences.md]
```

Multiple Related Rules (Merged)

```
1 Toolset (path ~/.local/bin/, details → ~/.hermes/config/tools-
2 reference.md):
3 - translate: when user requests translation
4 - ocr: when image with text is received
5 - weather: when user asks about weather
6 - email: when user requests email operations
```

System-Critical Rules (No Compression)

```
1 Hermes installation: ~/.local/bin/hermes, source code ~/hermes-agent
2 Dashboard port: 9119, Profile: default=User1, profile2=User2
```

Evaluation Scoring System

Use this formula to decide whether to keep, compress, or migrate each Memory entry:

```
1 Score = (Usage Frequency × 2) + (System Criticality × 2) + (Entry Length
2 × 1)
```

Dimension	Score 3	Score 2	Score 1
Usage Frequency	Every session	Occasionally	Rarely
System Criticality	Tool failure without it	Quality impact	Convenience only
Entry Length	150 chars	50-150 chars	≤50 chars

Decision Matrix:

Score	Action	Example
≥8	Keep original text	Installation paths, service ports
6-7	Compress to one-line index	Tool triggers, preference rules
≤5	Migrate to detail file	API credentials, full workflows

Implementation Guide

Step 1: Inventory Current Memory

Create a spreadsheet or list of all Memory and User Profile entries:

Entry	Chars	Frequency	Criticality	Length	Score	Action
Hermes installation path	45	3	3	1	13	KEEP
Translation tool config	180	2	2	3	11	KEEP→COMPRESS
Travel planning preferences	320	1	1	3	7	MIGRATE

Step 2: Create Detail Files Directory

```
1 mkdir -p ~/.hermes/config
```

Recommended file structure:

```
1 ~/.hermes/config/
2 |— api-services.md      # API credentials, external services
3 |— workflow-preferences.md # Workflows, user preferences
4 |— tools-reference.md   # Tool commands, trigger conditions
5 |— system-config.md     # Environment variables, ports, paths
```

Step 3: Migrate Entries

For each entry that needs migration:

1. **Write to detail file** under appropriate section

2. **Replace Memory entry** with index format
3. **Verify** the new index triggers correctly

Example migration for translation tool:

```
1 # 1. Write full config to detail file
2 cat >> ~/.hermes/config/api-services.md << 'EOF'
3
4 ## Translation Tool
5 - Purpose: Text translation
6 - Path: /opt/tools/translator/
7 - Command: translate "text" --to en
8 - Credentials: API Key=REDACTED, App ID=REDACTED
9 - Supported languages: 200+
10 - Trigger condition: When user requests translation
11 - Notes: Auto-detect source language
12 EOF
13
14 # 2. Update Memory (via Hermes memory tool)
15 # memory(action="replace", old_text="Translation CLI...",
16 #         content="Toolset: translate, ocr, weather, email [→ ~/.hermes/
17 #         config/api-services.md]")
```

Step 4: Validate Migration

Test each migrated rule:

```
1 # Test 1: Trigger condition
2 User: "Translate this text to English"
3 Expected: AI calls translate command
4
5 # Test 2: Direct inquiry
6 User: "What's your translation tool config?"
7 Expected: AI reads ~/.hermes/config/api-services.md
8
9 # Test 3: File modification
10 # Edit api-services.md, change API key
11 # Next conversation should use new key
12
13 # Test 4: File deletion
14 # Remove api-services.md
15 # AI should detect invalid path and prompt recreation
```

Configuration Templates

[api-services.md](#)

```
1  # API Service Configuration
2
3  ## Translation Service
4  - Purpose: Text translation
5  - Path: /opt/tools/translator/
6  - Command: translate "text" --to en
7  - Credentials: API Key=REDACTED, App ID=REDACTED
8  - Supported languages: 200+
9  - Trigger condition: When user requests translation
10 - Notes: Auto-detect source language
11
12 ## OCR Service
13 - Purpose: Image text recognition
14 - Path: /opt/tools/ocr/
15 - Command: ocr image.jpg
16 - Credentials: API Key=REDACTED, Secret Key=REDACTED
17 - Trigger condition: When image with text is received
18 - Notes: Supports Chinese/English mixed recognition
19
20 ## Weather Service
21 - Purpose: Weather queries
22 - Path: /opt/tools/weather/
23 - Command: weather "city name"
24 - Credentials: API Key=REDACTED
25 - Trigger condition: When user asks about weather
26 - Notes: Returns 3-day forecast
```

[workflow-preferences.md](#)

```
1  # Workflows and User Preferences
2
3  ## Text-to-Speech Strategy
4  - Rule: Analyze text emotion first, then generate style instructions
5  - Trigger condition: When TTS is requested
6  - Flow:
7      1. Analyze text emotion (happy/sad/neutral)
8      2. Generate corresponding style instructions
9      3. Read by emotion segments
10 - Example: Poetry reading needs to distinguish lyrical and narrative
11 sections
```

```

12
13 ## Subtitle Processing Flow
14 - Rule: Generate monolingual source subtitles first, then
    translate separately
15 - Trigger condition: When video/audio files are sent
16 - Flow:
17     1. Generate source language subtitles with Whisper
18     2. Check timeline accuracy
19     3. Translate subtitle text separately
20     4. Merge into bilingual subtitles
21 - Note: Clean up temporary files after completion
22
23 ## Courseware Extraction Standards
24 - Rule: Extract all concepts, definitions, key information
25 - Trigger condition: When PDF/Word courseware is sent
26 - Flow:
27     1. Extract text content
28     2. Identify concepts and definitions
29     3. Organize by topic structure
30     4. Generate structured notes
31 - Format: Use Markdown heading levels

```

tools-reference.md

```

1 # Tools Reference Manual
2
3 ## Translation Tool (translate)
4 - Purpose: Multi-language text translation
5 - Command: translate "text to translate" --to en
6 - Trigger condition: When user requests translation
7 - Credentials: Built-in script (api-services.md)
8 - Supported languages: 200+ languages
9 - Example: translate "Hello world" --to zh → "你好世界"
10
11 ## OCR Tool (ocr)
12 - Purpose: Image text recognition
13 - Command: ocr <image path>
14 - Trigger condition: When image with text is received
15 - Credentials: Built-in script (api-services.md)
16 - Supported formats: JPG, PNG, BMP
17 - Example: ocr /tmp/photo.jpg
18
19 ## Weather Query (weather)
20 - Purpose: Real-time weather and forecasts
21

```



```

22 - Command: weather "city name"
23 - Trigger condition: When user asks about weather
24 - Credentials: Built-in script (api-services.md)
25 - Returns: Temperature, humidity, wind, weather conditions
26 - Example: weather "Beijing" → "Sunny, 25°C"
27
28 ## Email Tool (email)
29 - Purpose: Email management
30 - Commands:
31   - email inbox          # View inbox
32   - email read <id>      # Read email
33   - email send <to> <subject> <body>
34   - email search <keyword>
35 - Trigger condition: When user requests email operations
36 - Credentials: Built-in script (api-services.md)

```

system-config.md

```

1  # System Configuration
2
3  ## Hermes Installation
4  - Main program: ~/.local/bin/hermes
5  - Source code: ~/hermes-agent
6  - Virtual environment: ~/hermes-agent/venv
7  - HERMES_HOME: ~/.hermes
8
9  ## Service Ports
10 - Dashboard: 9119 (default Profile)
11 - Dashboard: 9229 (secondary Profile)
12 - Gateway: 8080
13
14 ## Profile Mapping
15 - default → Primary User (port 9119)
16 - profile2 → Secondary User (port 9229)
17
18 ## Toolset Status
19 Enabled: web, terminal, file, code_execution, tts, skills, todo,
memory,
20         session_search, clarify, delegation, cronjob, messaging
21 Disabled: computer_use, image_gen, browser, vision
22
23 ## Current Model
24 - Provider: custom
25 - Model: your-model-name

```

Real-World Case Study

Case: Multi-Tool Development Environment

A developer using Hermes Agent for daily development accumulated **15+ tool configurations** over 3 months:

Before optimization:

- Memory usage: 89% of context window
- Frequent context truncation during complex tasks
- Agent occasionally "forgot" tool configurations mid-conversation
- ~6,000 characters injected every turn regardless of task

After implementing Memory Index Pattern:

- Memory usage: 52% of context window
- All rules preserved and accessible
- Smooth conversations even with complex multi-tool workflows
- ~2,500 characters injected per turn (58% reduction)

Key insight: The developer kept system-critical paths (installation directory, service ports) in Memory, but migrated all API credentials and workflow details to external files. The trigger conditions remained in Memory, ensuring the agent knew *when* to load each tool's details.

Performance Benchmarks

Token Consumption

Metric	Before	After	Improvement
Memory Usage	80-90%	45-60%	-30~40%
User Profile	70-80%	25-35%	-45~55%
Fixed Tokens/Turn	~6,000 chars	~2,500 chars	-58%
Available Context	~2,000 chars	~5,500 chars	+175%

Latency Impact

Operation	Latency	Notes
Index lookup	0ms	Already in Memory
Detail file read	~100ms	read_file() call
Total overhead	~100ms	Negligible for most use cases

Generalization Guide

Adapting to Other AI Agents

This pattern works with any agent that has:

1. **Persistent memory** injected into context
2. **File read capability** (or similar detail retrieval)
3. **Trigger-based execution** (conditional rule activation)

Generic Implementation Steps

```
1  # Pseudocode for any agent framework
2
3  class MemoryIndexPattern:
4      def __init__(self):
5          self.index_layer = {} # Rules in memory (always loaded)
6          self.detail_layer = {} # Details in files (loaded on-demand)
7
8      def add_rule(self, rule_id, trigger, tool, detail_path,
9          full_content):
10         # Store index in memory
11         self.index_layer[rule_id] = {
12             'trigger': trigger,
13             'tool': tool,
14             'detail_path': detail_path
15         }
16
17         # Store full content in file
18         write_file(detail_path, full_content)
```

```

19
20     def execute_rule(self, user_input):
21         # Find matching trigger
22         for rule_id, rule in self.index_layer.items():
23             if matches_trigger(user_input, rule['trigger']):
24                 # Load details only when needed
25                 details = read_file(rule['detail_path'])
26                 return execute_with_details(rule['tool'], details)
27
28     return None

```

Platform-Specific Adaptations

LangChain

```

1  from langchain.memory import ConversationBufferMemory
2  from langchain.tools import Tool
3
4  class IndexedMemory(ConversationBufferMemory):
5      def __init__(self, detail_dir="./config"):
6          super().__init__()
7          self.detail_dir = detail_dir
8          self.index = {} # {trigger: (tool_name, detail_file)}
9
10     def add_indexed_rule(self, trigger, tool_name, detail_file,
11 full_config):
12         self.index[trigger] = (tool_name, f"{self.detail_dir}/{
13 {detail_file}")
14         # Save full config to file
15         with open(f"{self.detail_dir}/{detail_file}", "w") as f:
16             f.write(full_config)
17
18     def get_relevant_context(self, user_input):
19         # Only return matching index entries
20         for trigger, (tool, path) in self.index.items():
21             if trigger.lower() in user_input.lower():
22                 return f"Use {tool}, details in {path}"
23         return ""

```

AutoGPT

```

1 // memory_index.json
2 {
3     "rules": [
4         {
5             "trigger": "user requests translation",
6             "tool": "translate",
7             "detail_file": "api-services.md",
8             "index_text": "Translation needed → use translate command →
~/.hermes/config/api-services.md"
9         }
10    ]
11 }

```

CrewAI

```

1 from crewai import Agent, Task
2
3 class IndexedAgent(Agent):
4     def __init__(self, *args, **kwargs):
5         super().__init__(*args, **kwargs)
6         self.memory_index = {}
7
8     def add_rule(self, trigger, tool, detail_path):
9         self.memory_index[trigger] = {
10             'tool': tool,
11             'detail_path': detail_path
12         }
13
14     def execute_task(self, task: Task):
15         # Check memory index first
16         for trigger, config in self.memory_index.items():
17             if trigger in task.description:
18                 details = self._load_details(config['detail_path'])
19                 return self._execute_with_details(config['tool'],
20 details, task)
21
22         return super().execute_task(task)

```

Migration Checklist for Other Platforms

- ☐ Identify memory/context injection mechanism
- ☐ Implement trigger detection (keyword matching or LLM classification)
- ☐ Create detail file storage system (local files, database, or cloud)
- ☐ Modify execution flow to load details on-demand

- ☐ Test trigger accuracy and detail retrieval latency
 - ☐ Document platform-specific index format
-

FAQ

Q: What if the detail file is deleted?

A: The agent detects an invalid path and prompts the user to recreate it. The index entry remains valid as a reference.

Q: How do I handle frequently changing credentials?

A: Store credentials in detail files only. Update the file when credentials change—no Memory modification needed.

Q: Can I use cloud storage for detail files?

A: Yes! Any accessible path works:

- Local files: `~/.hermes/config/api.md`
- Network mount: `/mnt/nas/hermes-config/api.md`
- Cloud sync: `~/Dropbox/hermes-config/api.md`

Q: What if I have too many detail files?

A: Keep 4-6 category files maximum. Over-fragmentation increases read operations and wastes tokens.

Q: Is this compatible with Hermes's built-in memory tools?

A: Fully compatible. Use `memory(action="replace")` to update index entries, and `read_file()` to access details.

Q: How do I handle rules that need both index and detail?

A: The index format already includes the detail path:

```
1 Trigger → Tool [→ ~/.hermes/config/detail.md]
```

The agent reads the detail file only when executing the rule.

Contributing

Reporting Issues

1. Check existing issues for duplicates
2. Include your Hermes version and configuration
3. Provide Memory usage before/after screenshots
4. Share anonymized detail file examples

Submitting Improvements

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/new-evaluation-metric`
3. Test with real Memory configurations
4. Submit a pull request with:
 - Before/after benchmarks
 - Migration guide updates
 - Template additions if applicable

Community Templates

Share your detail file templates:

```
1  # Submit your templates
2  cp ~/.hermes/config/api-services.md ./templates/your-name-api-
   services.md
3  git add templates/
4  git commit -m "Add API services template for [your service]"
5  git push origin feature/your-template
```

License

MIT License - see [LICENSE](#) for details.

Acknowledgments

- [Hermes Agent Team](#) for the memory framework
 - Community contributors who tested and refined the pattern
 - Users who shared their migration experiences
-

Changelog

v1.0.0 (2026-06-12)

- Initial release
- Core evaluation scoring system
- Four detail file templates
- Migration examples for API credentials and workflows
- Performance benchmarks

v1.1.0 (Planned)

- Automated migration script
 - LangChain/AutoGPT adapters
 - Cloud storage integration guide
 - Multi-language support for templates
-

Questions? Open an issue or join the [Hermes Agent Discord](#).

Found this helpful? Star the repo and share with other Hermes users!